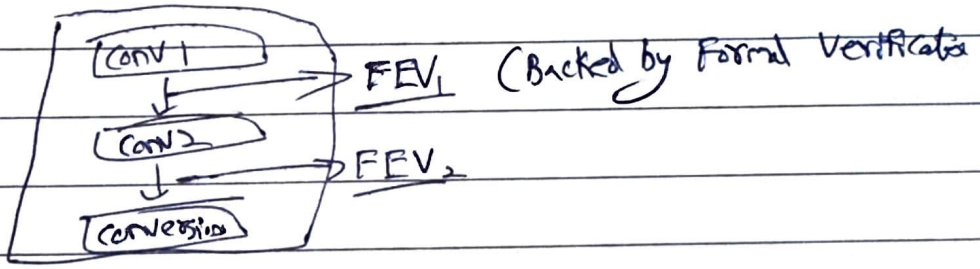
GSOC25 TL-Verilog Refactoring & AI Driven Conversion Project

→ Here Conversion → Semi-automated (Verilog to TL-Verilog u
LLMs)

Goal :-

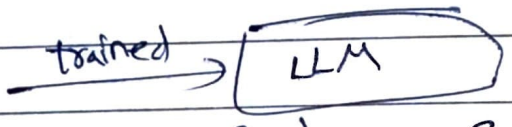
1) Thoughtful Approach AI can help

2)



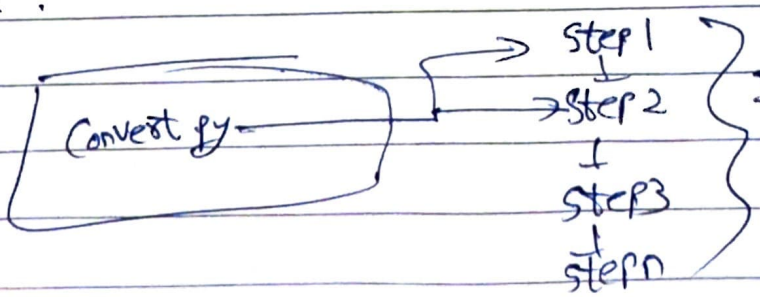
Approach :

1) Use various versions of chatgpt via its API
(Note :- We are not tuning a custom LLM that might be an option in future)



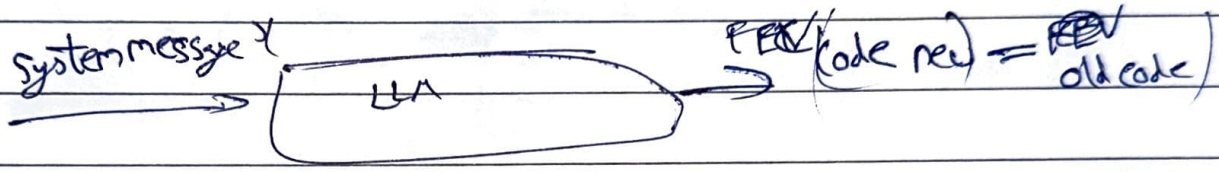
output (LLM) y = System messages (x)

→ Here Convert.py (script) → Controls the interaction with LLM.



Redwood EDA's SandPiper TM Tool → ifdess → refactoring → Verilog to System Verilog Look alike Code.

a) Each Step → Provides LLM with 'System message'



File:- default_system_message.txt (Some ex. msgs that define nature of conversion process)

b) Provide the prompt for the step and invoke LLM to do all (or) part of the step.

c) LLM response

d) Run FEV to check correctness (vs) previous version of code.

e) Iff FEV passes ==)

→ Update code

→ If the LLM gives flag_code update = 0, prompt for more more modification & run FEV.

↑ repeat (Do until Complete)

→ Next step & repeat the above.

Failure Scenario:-

→ At any step the process fails, the script will ask LLM (or) human assitance.

Storage:-

- output / updates → file system (output / updates)

→ This is needed so the script can be terminated / restarted. (It can pick where it left off)

Suppose → a proces has step1, step2, step3

→ A Script is terminated in blw after step2,

f. S.

updates until step 2 → As it stored the updates until step2 it can pick where it is terminated.

→ History of changes is maintained with an ability to revert

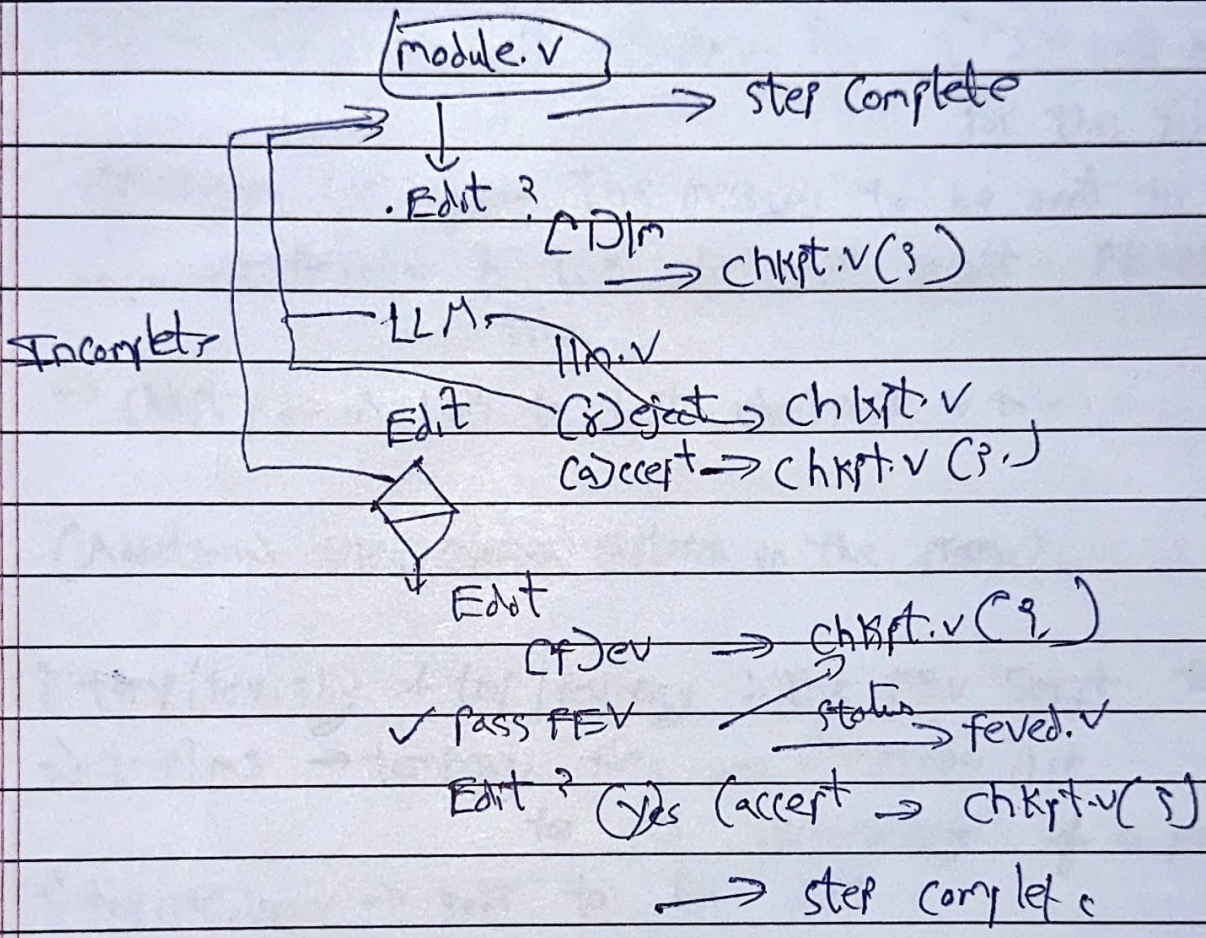
In future

files System / files 2 (Training data) → LLM

→ After refacting → Verilog can be converted to TL-Verilog & refactored the verilog further

Flow of Convert.py

Verilog Conversion process



→ There are 3 files in the process

1) module.v
2) → chkpt.v
3) feved.v

Status:

→ The Initial script is in place for the verilog conversion steps

Convertify (Reading)

Tools used

1) SymbiYosys (refactoring steps are formally verified.

<module_name>_orig.v

<module_name>.v WIP refactored file (FEV will be run for this file)

<messages.<pid>.json The messages to be sent to LLM.

-Current/feved.v A Link to most recent FEVed Verilog file.

→ Chkpt.v → A Link to Last checkpoint .v file

(Additional files, created/captured in the process)

1) tmp/fev.sby & tmp/fev.eqy :- The FEV Script for the
2) tmp/ms → temporary files used Conversion Ch. for MS preprocessing of a proj
3) tmp/pre_llm.v → sent to llm API
tmp/pre_llm_resp.v → Verilog response from LLM
tmp/diff.v Diff (llm.v & llm_resp.v)

4) tmp/diff_mod.v → a modification of diff.v to ignore
5) tmp/llm_upd.v → update verilog file after applying diff_mod.v.

6) tmp/llm.v → The updated Verilog (llm_upd.v (or) prech
→ llm_response.txt :- The LLM response file

default_system_message.txt

Overview:

1) Refactoring requests are farmed out any available agent.
   → Each request is bundled as an isolated job providing you with necessary background
      a) background: background information that is relevant to the refactoring ster
      b) prompt
      c) verilog

Several roles
→ Agents, Automation Software, FEV, Human-user